

06 | 复合类型（下）：枚举与模式匹配

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们一起学习 Rust 中的枚举（enum）和模式匹配（pattern matching）。

枚举是 Rust 中非常重要的复合类型，也是最强大的复合类型之一，广泛用于属性配置、错误处理、分支流程、类型聚合等场景中。学习完这节课后，你会对 Rust 的地道风格有新的认识。

枚举：强大的复合类型

枚举是这样一种类型，它容纳选项的可能性，每一种可能的选项都是一个变体（variant）。Rust 中的枚举使用关键字 **enum** 定义，这点与 Java、C++ 都是一样的。与它们不同的是，Rust 中的枚举具有更强大的表达能力。

在 Rust 中，枚举中的所有条目被叫做这个枚举的变体。比如：

 复制代码

```
1 enum Shape {  
2     Rectangle,  
3     Triangle,  
4     Circle,  
5 }
```

定义了一个形状（Shape）枚举，它有三个变体：长方形 Rectangle、三角形 Triangle 和圆形 Circle。

枚举与结构体不同，**结构体的实例化需要所有字段一起起作用，而枚举的实例化只需要且只能是其中一个变体起作用。**

负载

Rust 中枚举的强大之处在于，enum 中的变体可以挂载各种形式的类型。所有其他类型，比如字符串、元组、结构体等等，都可以作为 enum 的负载（payload）被挂载到其中一个变体上。比如，扩展一下上面的代码示例。

 复制代码

```
1 enum Shape {  
2     Rectangle { width: u32, height: u32 },  
3     Triangle((u32, u32), (u32, u32), (u32, u32)),  
4     Circle { origin: (u32, u32), radius: u32 },  
5 }
```

我们给 Shape 枚举的三个变体都挂载了不同的负载。Rectangle 挂载了一个结构体负载表示宽和高的属性。

 复制代码

```
1 {width: u32, height: u32}
```

为了看得更清楚，你也可以单独定义一个结构体，然后把它挂载到 Rectangle 变体上。

 复制代码

```
1 struct Rectangle {
2     width: u32,
3     height: u32
4 }
5
6 enum Shape {
7     Rectangle(Rectangle),
8     // ...
9 }
```

Triangle 变体挂载了一个元组负载 `((u32, u32), (u32, u32), (u32, u32))`，表示三个顶点。

Circle 变体挂载了一个结构体负载 `{ origin: (u32, u32), radius: u32 }`，表示一个原点加半径长度。

枚举的变体能够挂载各种类型的负载，是 Rust 中的枚举超强能力的来源，你可以通过上面例子来细细品味 Rust 的这种表达力。enum 就像一个筐，什么都能往里面装。

为了让你更熟悉 Rust 的枚举表达形式，我再举一个例子。下面的示例中 WebEvent 表示浏览器里面的 Web 事件。

 复制代码

```
1 enum WebEvent {
2     PageLoad,
3     PageUnload,
4     KeyPress(char),
5     Paste(String),
6     Click { x: i64, y: i64 },
7 }
```

你可以表述出不同变体的意义，还有每个变体所挂载的负载类型吗？期待看到你的答案。

枚举的实例化

枚举的实例化实际是枚举变体的实例化。比如：

[复制代码](#)

```
1 let a = WebEvent::PageLoad;
2 let b = WebEvent::PageUnload;
3 let c = WebEvent::KeyPress('c');
4 let d = WebEvent::Paste(String::from("batman"));
5 let e = WebEvent::Click { x: 320, y: 240 };
```

可以看到，不带负载的变体实例化和带负载的变体实例化不一样。带负载的变体实例化要根据不同变体附带的类型做特定的实例化。

类 C 枚举

Rust 中也可以定义类似 C 语言中的枚举。

示例：

[复制代码](#)

```
1 // 给枚举变体一个起始数字值
2 enum Number {
3     Zero = 0,
4     One,
5     Two,
6 }
7
8 // 给枚举每个变体赋予不同的值
9 enum Color {
10     Red = 0xff0000,
11     Green = 0x00ff00,
12     Blue = 0x0000ff,
13 }
14
15 fn main() {
16     // 使用 as 进行类型的转化
17     println!("zero is {}", Number::Zero as i32);
18     println!("one is {}", Number::One as i32);
19
20     println!("roses are #{:06x}", Color::Red as i32);
21     println!("violets are #{:06x}", Color::Blue as i32);
22 }
```



```
23 // 输出
24 zero is 0
25 one is 1
26 roses are #ff0000
27 violets are #0000ff
```

可以看到，我们能够像 C 语言那样，在定义枚举变体的时候，指定具体的值。这在底层系统级开发、协议栈开发、嵌入式开发的场景会经常用到。

打印的时候，只需要使用 `as` 操作符将变体转换为具体的数值类型即可。

代码中的 `println!` 里的 `{:06x}` 是格式化参数，这里表示打印出值的 16 进制形式，占位 6 个宽度，不足的用 0 补齐。你可以顺便了解一下 `println` 打印语句中 [🔗 格式化参数](#) 的详细内容。格式化参数相当丰富，我们可以在以后不断地实践中去熟悉和掌握它。

空枚举

Rust 中也可以定义空枚举。比如 `enum MyEnum {}`；。它其实与单元结构体一样，都表示一个类型。但是它不能被实例化。目前看起来好像没什么作用，我们只需要了解这种表示形式就可以了。

[📄 复制代码](#)

```
1 enum Foo {}
2
3 let a = Foo {}; // 错误的
4
5 // 提示
6 expected struct, variant or union type, found enum `Foo`
7 not a struct, variant or union type
```

impl 枚举

Rust 有个关键字 **impl** 可以用来给结构体或其他类型实现方法，也就是关联在某个类型上的函数。——第 5 讲

枚举同样能够被 impl。比如：

[复制代码](#)

```
1 enum MyEnum {
2     Add,
3     Subtract,
4 }
5
6 impl MyEnum {
7     fn run(&self, x: i32, y: i32) -> i32 {
8         match self {                                // match 语句
9             Self::Add => x + y,
10            Self::Subtract => x - y,
11        }
12    }
13 }
14
15 fn main() {
16     // 实例化枚举
17     let add = MyEnum::Add;
18     // 实例化后执行枚举的方法
19     add.run(100, 200);
20 }
```

但是不能对枚举的变体直接 impl。

[复制代码](#)

```
1 enum Foo {
2     AAA,
3     BBB,
4     CCC
5 }
6
7 impl Foo::AAA {    // 错误的
8 }
```

一般情况下，枚举会用来做配置，并结合 match 语句使用来做分支管理。**如果要定义一个新类型，在 Rust 中主要还是使用结构体。**

match

接下来我们开始学习和枚举搭配使用的 `match` 语句。

match + 枚举

其实在上面的示例中，就已经出现 `match` 关键字了。它的作用是判断或匹配值是哪一个枚举的变体。下面我们看一个例子。

[复制代码](#)

```
1  #[derive(Debug)]
2  enum Shape {
3      Rectangle,
4      Triangle,
5      Circle,
6  }
7
8  fn main() {
9      let shape_a = Shape::Rectangle; // 创建实例
10     match shape_a {                // 匹配实例
11         Shape::Rectangle => {
12             println!("{:?}", Shape::Rectangle); // 进了这个分支
13         }
14         Shape::Triangle => {
15             println!("{:?}", Shape::Triangle);
16         }
17         Shape::Circle => {
18             println!("{:?}", Shape::Circle);
19         }
20     }
21 }
22 // 输出
23 Rectangle
```

你可以试着改变实例为另外两种变体，看看打印出的信息有没有变化，然后判断上面的代码走了哪个分支。

match 可返回值

就像大多数 Rust 语法一样，`match` 语法也是可以有返回值的，所以也叫做 `match` 表达式，我们来看一下示例。

```
1 #[derive(Debug)]
2 enum Shape {
3     Rectangle,
4     Triangle,
5     Circle,
6 }
7
8 fn main() {
9     let shape_a = Shape::Rectangle; // 创建实例
10    let ret = match shape_a {        // 匹配实例，并返回结果给ret
11        Shape::Rectangle => {
12            1
13        }
14        Shape::Triangle => {
15            2
16        }
17        Shape::Circle => {
18            3
19        }
20    };
21    println!("{}", ret);
22 }
23 // 输出
24 1
```

因为 `shape_a` 被赋值为 `Shape::Rectangle`，所以程序匹配到第一个分支并返回 1，变量 `ret` 的值为 1。

```
1 let ret = match shape_a {
```

这种写法就是比较地道的 Rust 写法，可以让代码显得更紧凑。

注意，**match 表达式中各个分支返回的值的类型必须相同。**

所有分支都必须处理

match 表达式里所有的分支都必须处理，不然 Rustc 小助手会拦住你，不让你通过。这是怎么回事呢？你可以看一下示例代码。

[复制代码](#)

```

1  #[derive(Debug)]
2  enum Shape {
3      Rectangle,
4      Triangle,
5      Circle,
6  }
7
8  fn main() {
9      let shape_a = Shape::Rectangle; // 创建实例
10     let ret = match shape_a {        // 匹配实例
11         Shape::Rectangle => {
12             1
13         }
14         Shape::Triangle => {
15             2
16         }
17         // Shape::Circle => {
18             // 3
19         // }
20     };
21     println!("{}", ret);
22 }
```

上面这段代码在编译的时候会出错。

[复制代码](#)

```

1  error[E0004]: non-exhaustive patterns: `Shape::Circle` not covered
2    --> src/main.rs:10:19
3    |
4  10 |     let ret = match shape_a {                // 匹配实例
5    |                                ^^^^^^^^^ pattern `Shape::Circle` not covered
6    |
7  note: `Shape` defined here
8    --> src/main.rs:5:3
9    |
10  2 |     enum Shape {
11    |         -----
12  ...
13  5 |         Circle,
```

```

14 |     ^^^^^^ not covered
15 |     = note: the matched value is of type `Shape`
16 | help: ensure that all possible cases are being handled by adding a match arm with
17 |
18 | 16 ~     },
19 | 17 +     Shape::Circle => todo!()
20 |

```

小助手提示说，`Shape::Circle` 分支没有覆盖到，不允许通过，然后直接贴心地给出了修改建议！Rustc 小助手如此贴心，这种保姆级服务是你在 Java、C++ 等其他语言中感受不到的。

_ 占位符

有时，你确实想测试一些东西，或者就是不想处理一些分支，可以用 `_` 偷懒。

比如上面代码可以修改成这样：

[复制代码](#)

```

1  #[derive(Debug)]
2  enum Shape {
3      Rectangle,
4      Triangle,
5      Circle,
6  }
7
8  fn main() {
9      let shape_a = Shape::Rectangle;
10     let ret = match shape_a {
11         Shape::Rectangle => {
12             1
13         }
14         _ => {
15             10
16         }
17     };
18     println!("{}", ret);
19 }

```

相当于除 `Shape::Rectangle` 之外的分支我们都统一用 `_` 占位符进行了处理了。

更广泛的分支

match 除了配合枚举进行分支管理外，还可以与其他基础类型结合进行分支分派。我们可以看一个 The Book 里的示例。

[复制代码](#)

```
1 fn main() {
2     let number = 13;
3     // 你可以试着修改上面的数字值，看看下面走哪个分支
4
5     println!("Tell me about {}", number);
6     match number {
7         // 匹配单个数字
8         1 => println!("One!"),
9         // 匹配几个数字
10        2 | 3 | 5 | 7 | 11 => println!("This is a prime"),
11        // 匹配一个范围，左闭右闭区间
12        13..=19 => println!("A teen"),
13        // 处理剩下的情况
14        _ => println!("Ain't special"),
15    }
16 }
```

可以看到，match 可以用来匹配一个具体的数字、一个数字的列表，或者一个数字的区间等等，非常灵活。在这点上，可比 C、C++，或者 Java 的 `switch .. case` 灵活多了。

模式匹配

match 实际是模式匹配的入口，从 match 表达式我们可引出模式匹配的概念。模式匹配就是**按对象值的结构**进行匹配，并且可以取出符合模式的值。下面我们通过一些示例来解释这句话。

模式匹配不限于在 match 中使用。除了 match 外，Rust 还给模式匹配提供了其他一些语法层面的设施。

if let

当要匹配的分支只有两个或者在这个位置只想先处理一个分支的时候，可以直接用 if let。

比如下面这段代码就可以使用 if let。

[复制代码](#)

```
1 let shape_a = Shape::Rectangle;
2 match shape_a {
3     Shape::Rectangle => {
4         println!("1");
5     }
6     _ => {
7         println!("10");
8     }
9 };
```

改写为：

[复制代码](#)

```
1 let shape_a = Shape::Rectangle;
2 if let Shape::Rectangle = shape_a {
3     println!("1");
4 } else {
5     println!("10");
6 }
```

是不是相比于 match，使用 if let 的代码量有所简化？

while let

while 后面也可以跟 let，实现模式匹配。比如：

[复制代码](#)

```
1 #[derive(Debug)]
2 enum Shape {
3     Rectangle,
4     Triangle,
5     Circle,
6 }
7
8 fn main() {
9     let mut shape_a = Shape::Rectangle;
```

```

10     let mut i = 0;
11     while let Shape::Rectangle = shape_a {    // 注意这一句
12         if i > 9 {
13             println!("Greater than 9, quit!");
14             shape_a = Shape::Circle;
15         } else {
16             println!("`i` is `{:?}`. Try again.", i);
17             i += 1;
18         }
19     }
20 }
21 // 输出
22 `i` is `0`. Try again.
23 `i` is `1`. Try again.
24 `i` is `2`. Try again.
25 `i` is `3`. Try again.
26 `i` is `4`. Try again.
27 `i` is `5`. Try again.
28 `i` is `6`. Try again.
29 `i` is `7`. Try again.
30 `i` is `8`. Try again.
31 `i` is `9`. Try again.
32 Greater than 9, quit!

```

上面示例构造了一个 while 循环，手动维护计数器 i，递增到 9 之后，退出循环。

看起来，在条件判断语句那里用 `while Shape::Rectangle == shape_a` 也行，好像用 `while let` 的意义不大。我们来试一下，编译之后，报错了。

 复制代码

```

1 error[E0369]: binary operation `==` cannot be applied to type `Shape`

```

说 `==` 号不能作用在类型 `Shape` 上，你可以思考一下为什么。

如果一个枚举变体带负载，使用模式匹配可以把这个负载取出来，这点就比较方便了，下面我们使用带负载的枚举来举例。

let

let 本身就支持模式匹配。其实前面的 if let、while let 本身使用的就是 let 模式匹配的能力。

[复制代码](#)

```
1  #[derive(Debug)]
2  enum Shape {
3      Rectangle {width: u32, height: u32},
4      Triangle,
5      Circle,
6  }
7
8  fn main() {
9      // 创建实例
10     let shape_a = Shape::Rectangle {width: 10, height: 20};
11     // 模式匹配出负载内容
12     let Shape::Rectangle {width, height} = shape_a else {
13         panic!("Can't extract rectangle.");
14     };
15     println!("width: {}, height: {}", width, height);
16 }
17
18 // 输出
19 width: 10, height: 20
```

在这个示例中，我们利用模式匹配解开了 shape_a 中带的负载（结构体负载），同时定义了 width 和 height 两个局部变量，并初始化为枚举变体的实例负载的值。这两个局部变量在后续的代码块中可以使用。

注意第 12 行代码。

[复制代码](#)

```
1  let Shape::Rectangle {width, height} = shape_a else {
```

这种语法是匹配结构体负载，获取字段值的方式。

匹配元组

元组也可以被匹配，比如下面这个例子。

 复制代码

```
1 fn main() {
2     let a = (1,2,'a');
3
4     let (b,c,d) = a;
5
6     println!("{:?}", a);
7     println!("{}", b);
8     println!("{}", c);
9     println!("{}", d);
10 }
```

这种用法叫做元组的析构，常用来从函数的多个返回值里取出数据。

 复制代码

```
1 fn foo() -> (u32, u32, char) {
2     (1,2,'a')
3 }
4
5 fn main() {
6     let (b,c,d) = foo();
7
8     println!("{}", b);
9     println!("{}", c);
10    println!("{}", d);
11 }
```

匹配枚举

前面已经讲过如何使用 let 把枚举里变体的负载解出来，这里我们再来看一个例子。

 复制代码

```
1 struct Rectangle {
2     width: u32,
3     height: u32
4 }
5
6 enum Shape {
7     Rectangle(Rectangle),
8     Triangle((u32, u32), (u32, u32), (u32, u32)),
9     Circle { origin: (u32, u32), radius: u32 },
```

```

10 }
11
12 fn main() {
13     let a_rec = Rectangle {
14         width: 10,
15         height: 20,
16     };
17
18     // 请打开下面这一行进行实验
19     //let shape_a = Shape::Rectangle(a_rec);
20     // 请打开下面这一行进行实验
21     //let shape_a = Shape::Triangle((0, 1), (3,4), (3, 0));
22
23     let shape_a = Shape::Circle { origin: (0, 0), radius: 5 };
24
25     // 这里演示了在模式匹配中将枚举的负载解出来的各种形式
26     match shape_a {
27         Shape::Rectangle(a_rec) => { // 解出一个结构体
28             println!("Rectangle {}, {}", a_rec.width, a_rec.height);
29         }
30         Shape::Triangle(x, y, z) => { // 解出一个元组
31             println!("Triangle {:?}, {:?}, {:?}", x, y, z);
32         }
33         Shape::Circle {origin, radius} => { // 解出一个结构体的字段
34             println!("Circle {:?}, {:?}", origin, radius);
35         }
36     }
37 }
38 // 输出
39 Circle (0, 0), 5

```

这个示例展示了如何将变体中的结构体整体、元组各部分、结构体各字段解析出来的方式。

用这种方式，我们可以在做分支处理的时候，顺便处理携带的信息，让代码变得相当紧凑而有意义（高内聚）。你需要熟悉并掌握这些写法，这样写起 Rust 代码来才会更加顺手。

匹配结构体

下面我们再看一个例子，了解结构体字段匹配过程中的一个细节。

 复制代码

```

1 #[derive(Debug)]
2 struct User {

```

```

3     name: String,
4     age: u32,
5     student: bool
6 }
7
8 fn main() {
9     let a = User {
10         name: String::from("mike"),
11         age: 20,
12         student: false,
13     };
14     let User {
15         name,
16         age,
17         student,
18     } = a;
19
20     println!("{}", name);
21     println!("{}", age);
22     println!("{}", student);
23     println!("{:?}", a);
24 }

```

编译输出:

 复制代码

```

1 error[E0382]: borrow of partially moved value: `a`
2   --> src/main.rs:24:22
3   |
4 16 |         name,
5   |         ---- value partially moved here
6   ...
7 24 |     println!("{:?}", a);
8   |                        ^ value borrowed here after partial move
9   |
10  = note: partial move occurs because `a.name` has type `String`, which does not
11  = note: this error originates in the macro `crate::format_args_nl` which come
12 help: borrow this binding in the pattern to avoid moving the value
13   |
14 16 |         ref name,
15   |         +++

```

编译提示出错了，在模式匹配的过程中发生了 partially moved。关于 partially moved 我们在上节课已经讲过。模式匹配过程中新定义的三个变量 name、age、student 分别得到了对应 User 实例 a 的三个字段值的所有权。

age 和 student 采用了复制所有权的形式（参考 [🔗第 2 讲移动还是复制部分](#)），而 name 字符串值则是采用了移动所有权的形式。a.name 被部分移动到了新的变量 name，所以接下来 a.name 就无法直接使用了。

这个示例说明 **Rust 中的模式匹配是一种释放原对象的所有权的方式**。

从 Rust 小助手的建议里我们看到了一个关键字：ref。

ref 关键字

Rustc AI 小助手建议我们添加一个关键字 ref，我们按它说的改改。

 复制代码

```
1  #[derive(Debug)]
2  struct User {
3      name: String,
4      age: u32,
5      student: bool
6  }
7
8  fn main() {
9      let a = User {
10         name: String::from("mike"),
11         age: 20,
12         student: false,
13     };
14     let User {
15         ref name,    // 这里加了一个ref
16         age,
17         student,
18     } = a;
19
20     println!("{}", name);
21     println!("{}", age);
22     println!("{}", student);
23     println!("{:?}", a);
24 }
```



```
25 // 输出
26 mike
27 20
28 false
29 User { name: "mike", age: 20, student: false }
```

可以看到，打印出了正确的值。

有些情况下，我们只是需要读取一下字段的值而已，不需要获得它的所有权，这时就可以通过 `ref` 这个关键字修饰符告诉 Rust 编译器，我现在只需要获得那个字段的引用，不要给我所有权。这就是 `ref` 出现的原因，用来**在模式匹配过程中提供一个额外的信息**。

使用了 `ref` 后，新定义的 `name` 变量的值其实是 `&a.name`，而不是 `a.name`，Rust 就不会再把所有权给 `move` 出来了，因此也不会发生 `partially moved` 这种事情，原来的 `User` 实例 `a` 还有效，因此就能被打印出来了。你可以体会一下其中的区别。

相应的，还有 `ref mut` 的形式。它是用于在模式匹配中获得目标的可变引用。

[复制代码](#)

```
1 let User {
2     ref mut name,    // 这里加了一个ref mut
3     age,
4     student,
5 } = a;
```

你可以做做实验体会一下。

Rust 中强大的模式匹配这个概念并不是 Rust 原创的，它来自于函数式语言。你如果感兴趣的话，可以了解一下 `Ocaml`、`Haskell` 或 `Scala` 中模式匹配的相关概念。

函数参数中的模式匹配

函数参数其实就是定义局部变量，因此模式匹配的能力在这里也能得到体现。

示例 1:

 复制代码

```
1 fn foo((a, b, c): (u32, u32, char)) { // 注意这里的定义
2     println!("{}", a);
3     println!("{}", b);
4     println!("{}", c);
5 }
6
7 fn main() {
8     let a = (1, 2, 'a');
9     foo(a);
10 }
```

上例，我们把元组 `a` 传入了函数 `foo()`，`foo()` 的参数直接定义成模式匹配，解析出了 `a`、`b`、`c` 三个元组元素的内容，并在函数中使用。

示例 2:

 复制代码

```
1 #[derive(Debug)]
2 struct User {
3     name: String,
4     age: u32,
5     student: bool
6 }
7
8 fn foo(User { // 注意这里的定义
9     name,
10    age,
11    student
12 }: User) {
13     println!("{}", name);
14     println!("{}", age);
15     println!("{}", student);
16 }
17
18 fn main() {
19     let a = User {
20         name: String::from("mike"),
21         age: 20,
22         student: false,
```

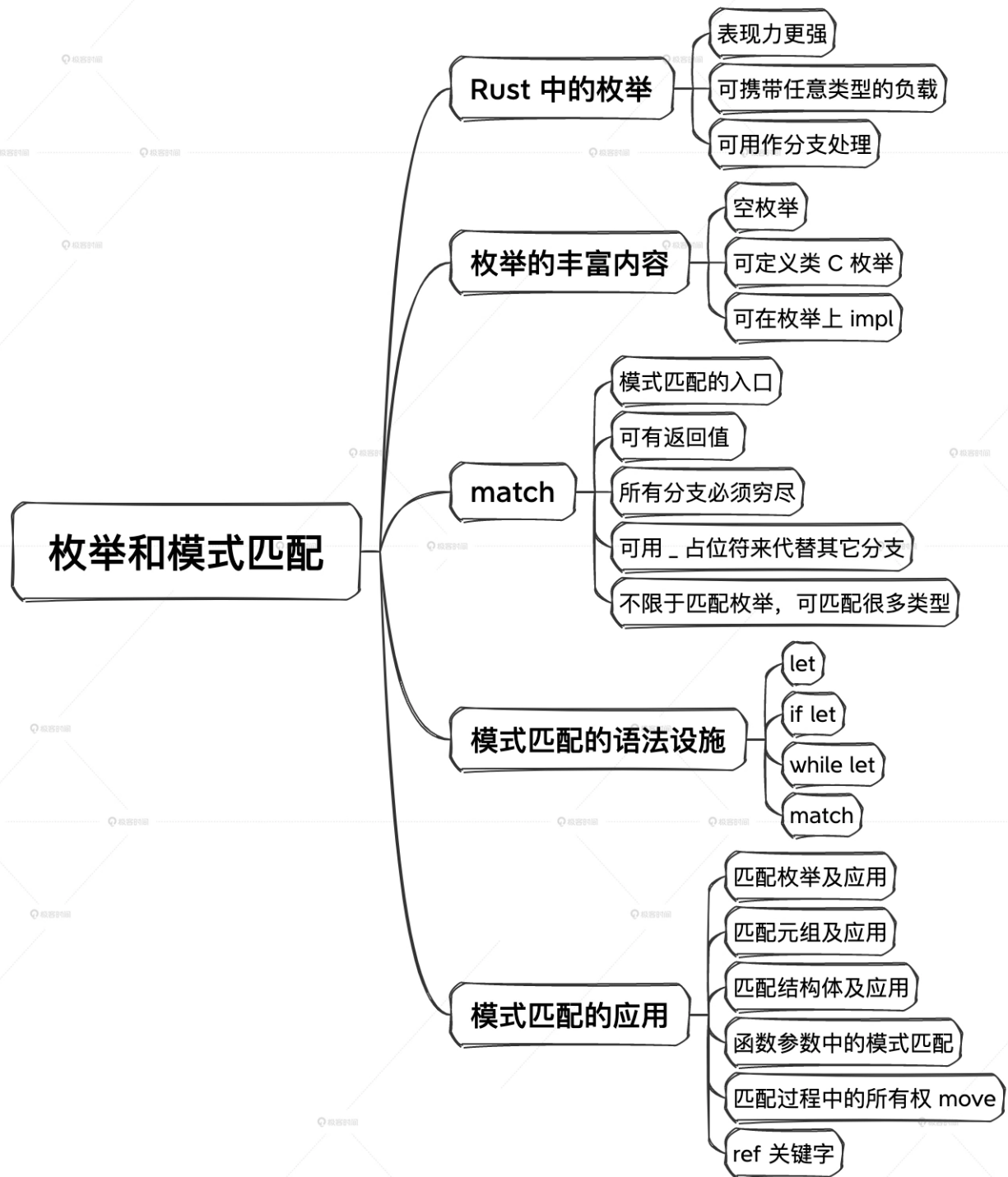
```
23     };  
24     foo(a);  
25 }
```

上例，我们把结构体 `a` 传入了函数 `foo()`，`foo()` 的参数直接定义成对结构体的模式匹配，解析出了 `name`、`age`、`student` 三个字段的内容，并在函数中使用。

小结

枚举是 Rust 中的重要概念，广泛用于属性配置、错误处理、分支流程、类型聚合等。在实际场景中，我们一般把结构体作为模型的主体承载，把枚举作为周边的辅助配置和逻辑分类。它们经常会搭配使用。

模式匹配是 Rust 里非常有特色的语言特性，我们在做分支逻辑处理的时候，可以通过模式匹配带上要处理的相关信息，还可以把这些信息解析出来，让代码的逻辑和数据内聚得更加紧密，让程序看起来更加赏心悦目。



思考题

match 表达式的各个分支中，如果有不同的返回类型的情况，应该如何处理？欢迎你在评论区留下自己的答案，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

参考资料

格式化参数：🔗 <https://shimo.im/outlink/gray?url=https%3A%2F%2Fdoc.rust-lang.org%2Fstd%2Ffmt%2Findex.html>

match 的语法规则：🔗 <https://doc.rust-lang.org/reference/expressions/match-expr.html>

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (25)

最新 精选



古明地觉

2023-11-02 来自中国台湾

老师，看到里面的一个例子，我产生了一些疑问。我先举个例子：

```

```
fn main() {
 let s = String::from("Hello");
 let p = &s;
 let s2 = *p;
}
```

```

如果直接将 `s` 赋值给 `s2`，那么毫无疑问会发生移动，`s` 不再有效。但如果是对 `s` 的引用进行解引用，那么编译器会提示无法移动，这是啥原因呀。我自己有一个猜测，因为 Rust 默认不会深度拷贝数据，所以如果 `let s2 = *p` 这条语句成立，就意味着要夺走 `s` 的所有权。但我们之所以要获取引用，就是为了不夺走原有变量 (`s`) 的所有权，于是在这种情况下，Rust 干脆提示不允许我们移动，除非它实现了 `Copy trait`，数据全部在栈上，浅拷贝之后数据彼此独立。

这样理解是正确的吗？Rust 的一些概念比较相似，容易出现混乱，所以想问问老师。

基于上面这个例子，再来看看文中的一个例子。

```

```
enum MyEnum {
 Add,
```

```
Subtract,
}
```

```
impl MyEnum {
 fn run(&self, x: i32, y: i32) -> i32 {
 // 这里的 self 显然是枚举的某个变体的引用
 // 因为 MyEnum 没有实现 Copy trait, 所以它和字符串一样, 不能通过解引用赋值
 // let obj = *self; 这里是不合法的, 由于不是可 Copy 的, 因此无法移动
 // 但问题来了, 为啥下面的代码是合法的, 原文是 match self, 但改成 match *self 也可以
 match *self {
 Self::Add => x + y,
 Self::Subtract => x - y,
 }
 }
}
```

因此这是我的第二个疑问, 为啥 `let obj = *self` 不合法, 但 `match *self` 就是合法的。

还有第三个疑问, 可能是受到 C 的影响, 因为变量和指针是无法比较的。所以在看到 `match self` 的一瞬间, 就忍不住试了一下 `match *self`, 因为参数是 `&self`, 所以 `self` 是枚举变体的引用。而 `Self::Add` 和 `Self::Subtract` 是具体的枚举变体, 它们之间比较总觉得有些别扭, 还是 `match *self` 看着顺眼。所以想问一下老师, 为啥这两者能够比较。

以上就是我的一些疑问, 还麻烦唐老师指导一下, Rust 的一些概念有点让人头晕。

作者回复: 非常用心的思考。反过来回答:

3. `match` 的时候, rust 做了自动解引用, 就是自动加了 `*self`。这点上 Rust 就是与 C 不同, Rust 有点猜测程序员的意图的味道。比如看如下代码, 也是类似做了自动解引用。

```
...

fn foo(a: &u32, b: &u32) {
 if a > b {
 println!("111");
 } else {
 println!("222");
 }
}
```

```
fn main() {
 foo(&5, &4);
}
...
```

2. match枚举时，这里不存在对负载的匹配捕获，因此不存在“再赋值”的操作，只是比对一下，于是跟它是Copy的还是Move的就没关系了。

1. 对，因为做了“再赋值”的操作。跟 Copy的还是Move的就有关系，所以就是你那样理解的。

Rust内部做了很多逻辑自洽的推理。

共 2 条评论 >

👍 6



### 一个人旅行

2023-11-01 来自北京

1. 默认情况下，struct不能进行比较，需要为Shape类型实现PartialEq trait特征。
2. 方式一：使用枚举，该枚举类型的枚举值表示一个类型。  
方式二：使用特征，所有的类型都需要实现该特征

作者回复: 可以的



👍 4



### 下雨天

2023-11-01 来自湖北

思考题：

```
pub enum Result<T1,T2>{
 V1(T1),
 V2(T2)
}
```

```
fn get_diff_value(number:i32) -> Result<i32,i32>{
 let r = match number {
 1 => Result::V1(1),
 _ => Result::V2(2)
 };
 r
}
```

```
fn main(){
 let r = get_diff_value(0);
 match r {
 Result::V1(x) => println!("V1={x}"),
 Result::V2(x) => println!("V2={x}")
 }
}
```

聚合的代价是不是需要解聚合。。。。

作者回复: 对，包，解包还是有点麻烦

共 2 条评论 >

👍 2



**水不要鱼**

2023-11-06 来自广东

老师，我有个问题，那个 ref name 那里是不是等于 &name，如果是的话，那不是相当于把一个 &String 赋值给了 String 么，还是说 rust 会自动用 &String 再去创建一个 String 然后给 name

作者回复: 直接给的引用出来，没有再创建一份新实例。



👍 1



**Michael**

2023-11-02 来自中国香港

上面的 ref 章节中，User.name 是 String 类型，为什么可以通过ref给它赋值 a.name 的引用

作者回复: 就是要得到引用才引入的ref

共 2 条评论 >

👍 1



**plh**

2023-11-01 来自四川

这一节 就值回票价了.

文中: [Rustc 小助手如此贴心，这种保姆级服务是你在 Java、C++ 等其他语言中感受不到的。]



个人感觉, Rustc 也像 大师样的,在编写代码的时候, 在人身边不厌其烦的指出潜在的问题,直到编译成功.

作者回复: 对的, 一个AI人类。



**Taozi**

2023-11-01 来自上海

1. enum变体可以挂载的类型是否就是对应了结构体的三种形态：单元结构体、元组结构体和命名结构体。
2. match表达式不同的返回类型可以使用enum进行聚合。

作者回复: 1. 可以这样理解。但要注意, enum中的选项名称本身是 变体, variant, 不是struct name;  
2. 对的。



**付佳伟**

2023-11-01 来自北京

思考题: match各个分支返回值类型不同时, 可以将各个类型实现同一个trait, 然后用Box将返回值包裹, 返回值类型定义Box<dyn trait>。不知道还有没有其他办法

作者回复: 你说这个方案后面会讲到。其它方案见其它同学说的有。



**HdUIprince**

2024-01-01 来自湖北

老师参考资料中的格式化参数的链接不太对?

作者回复: <https://doc.rust-lang.org/std/fmt/index.html> 这个,对的吧?



**tan**

2023-12-20 来自重庆

比java的枚举强大太多了

作者回复: 对的，Rust的枚举就是个多啦A梦的口袋。

